
Langflow-Bausteine

Bibliothekshackathon: KI-Workflows ohne Programmieren

Langflow 1.12.3 · September 2026

Gefördert durch:



Das KI-Servicezentrum Berlin-Brandenburg wird vom Bundesministerium für Forschung, Technologie und Raumfahrt unter dem Förderkennzeichen 16IS22092 gefördert.

Diese Anleitung beschreibt jeden Baustein (Komponente), den ihr während des Hackathons in Langflow nutzen könnt: was er tut, was hineingeht und was herauskommt. Die Bausteine stehen in derselben Reihenfolge wie in der Leiste **Components** links in Langflow: zuerst die Kategorien von oben nach unten, dann die Bausteine jeder Kategorie in alphabetischer Reihenfolge.

Sie gilt für Langflow 1.12.3, so wie es in diesem Projekt eingerichtet ist. Die Bausteine der Kategorie *Bibliothek* wurden für den Hackathon gebaut, alle anderen gehören zu Langflow.

Inhalt

1. So lest ihr diese Anleitung
2. Ein KI-Modell in Standard-Bausteinen nutzen
3. Bausteine
 - 3.1 Input & Output: Chat Input, Chat Output, Webhook
 - 3.2 Data Sources: API Request, Mock Data, SQL Database, URL, Web Search
 - 3.3 Models & Agents: A2A Agent, Agent, Embedding Model, Language Model, Message History, Prompt Template
 - 3.4 LLM Operations: Batch Run, Guardrails, LLM Selector, Smart Router, Smart Transform, Structured Output
 - 3.5 Files & Knowledge: File System, Knowledge, Memory Base, Read File, Write File
 - 3.6 Processing: Data Operations, Dynamic Create Data, Parser, Split Text, Type Convert
 - 3.7 Flow Control: Human Input, If-Else, Listen, Loop, Notify, Run Flow
 - 3.8 Utilities: Calculator, Current Date, Python Interpreter
 - 3.9 Bibliothek: Katalogsuche, KI-Modell, Literaturangaben prüfen, Literatursuche, Literaturverzeichnis finden, Quellen anreichern, Volltexte holen
4. Nicht in dieser Anleitung

1. So lest ihr diese Anleitung

Eingänge und Ausgänge. Eingänge sind am **linken** Rand eines Bausteins: Felder, die ihr ausfüllt, oder Punkte, an die ihr eine Linie anschließt. Ausgänge sind die Punkte am **rechten** Rand. Eine Linie führt immer von einem Ausgang zu einem Eingang. Viele Eingänge kann man entweder eintippen oder verbinden; sobald eine Linie angeschlossen ist, steht im Feld *Receiving input*.

Datentypen. Jeder Eingang und Ausgang hat einen Typ, und nur passende Typen lassen sich verbinden. Fahrt mit der Maus über einen Punkt, um seinen Typ zu sehen.

Typ	Was es ist
Message	Text, z. B. eine Chat-Nachricht, ein Prompt oder die Antwort eines Modells
JSON	Ein strukturierter Datensatz aus benannten Feldern, z. B. die Antwort eines Webdienstes. Ältere Langflow-Versionen nennen ihn <i>Data</i> .
Table	Zeilen und Spalten wie in einer Tabellenkalkulation. Ältere Versionen nennen sie <i>DataFrame</i> .
Language Model	Ein eingerichtetes KI-Modell, das ein anderer Baustein nutzen kann, z. B. ein Agent
Tool	Ein Baustein, den ein Agent von sich aus aufrufen darf (siehe <i>Tool Mode</i> unten)
Embeddings	Ein Modell, das Texte in Zahlen umwandelt, damit sich ähnliche Texte finden lassen. Nur für <i>Knowledge</i> nötig.

In den Tabellen unten nennt die Typ-Spalte auch einfache Einstellungen: *Text*, *Zahl*, *Schalter* (an/aus), *Auswahl* (ein Aufklappmenü oder Reiter), *Schieberegler*, *Datei* und *Liste*.

Versteckte Einstellungen. Viele Bausteine haben mehr Einstellungen, als sie zeigen. Klickt auf einen Baustein und dann in der kleinen Leiste darüber auf **Parameters**, um sie ein- oder auszublenden. Diese Anleitung nennt die versteckten Einstellungen, die nützlich sind; die übrigen könnt ihr so lassen.

Tool Mode. In derselben Leiste über einem Baustein gibt es den Schalter **Tool Mode**. Ist er an, bekommt der Baustein einen Ausgang *Toolset*, den ihr mit dem Eingang *Tools* eines Agenten verbindet. Der Agent entscheidet dann selbst, wann er den Baustein nutzt. Fast jeder Baustein lässt sich in den Tool Mode schalten; die Einträge unten erwähnen es, wo es besonders nützlich ist.

Beispiele. Kommt ein Baustein in einem der Beispiel-Flows im *Starter Project* vor, steht beim Eintrag, in welchem. Dort könnt ihr ihn in Aktion sehen.

2. Ein KI-Modell in Standard-Bausteinen nutzen

Mehrere Standard-Bausteine brauchen ein KI-Modell: Agent, Batch Run, Guardrails, LLM Selector, Smart Router, Smart Transform und Structured Output. Sie haben ein Feld **Language Model** mit einer Liste von Anbietern wie OpenAI. Diese Anbieter sind in diesem Projekt **nicht eingerichtet**.

Setzt stattdessen einen Baustein **KI-Modell** (Kategorie *Bibliothek*) auf die Arbeitsfläche und verbindet seinen Ausgang **Language Model** mit dem Eingang *Language Model* des Bausteins. Das Modell kommt dann vom Cluster oder von Ollama auf eurem Laptop, genau wie in der `.env` eingestellt. Flow 04 macht das für seinen Agenten.

Der Standard-Baustein **Language Model** selbst funktioniert wie KI-Modell, braucht aber einen Anbieter; nimmt stattdessen KI-Modell.

3. Bausteine

Die Bausteine in der Reihenfolge der Leiste, mit einem Unterabschnitt pro Kategorie.

3.1 Input & Output

3.1.1 Chat Input

Nimmt den Text, den ihr im **Playground** eintippt, und gibt ihn in den Flow. Fast jeder Flow beginnt damit.

	Name	Typ	Was es ist
Ein	Input Text	Text	Die Nachricht. Normalerweise leer lassen: Sie kommt aus dem Playground.
Aus	Chat Message	Message	Eure Nachricht, zum Verbinden mit einem Prompt, einem Modell oder einem anderen Baustein

Versteckte Einstellungen: *Files* (Dateien an die Nachricht anhängen), *Store Messages* (die Nachricht im Chatverlauf speichern), *Sender Name*.

Beispiele: alle Beispiel-Flows.

3.1.2 Chat Output

Zeigt ein Ergebnis im **Playground** an. Ein Flow kann mehrere Chat Outputs haben; jeder erscheint als eigene Nachricht.

	Name	Typ	Was es ist
Ein	Inputs	Message, JSON oder Table	Was angezeigt werden soll. Tabellen erscheinen als Tabellen.
Aus	Output Message	Message	Derselbe Inhalt noch einmal, falls ihr ihn weitergeben wollt

Versteckte Einstellungen: *Sender Name* (der Name über der Nachricht, standardmäßig „AI“), *Data Template* (wie JSON in Text umgewandelt wird).

Beispiele: alle Beispiel-Flows. Flow 01 nutzt zwei, einen für den Bericht und einen für die Tabelle.

3.1.3 Webhook

Lässt ein anderes Programm den Flow über das Internet starten, indem es Daten an eine Webadresse schickt. Für den Hackathon nicht nötig.

	Name	Typ	Was es ist
Ein	Endpoint	Text	Die Adresse, an die andere Programme Daten schicken (wird automatisch ausgefüllt)
Aus	JSON	JSON	Die geschickten Daten

3.2 Data Sources

3.2.1 API Request

Holt Daten von einem Webdienst (einer API), indem er dessen Adresse aufruft. So bindet ihr Datenbanken an, für die es keinen fertigen Baustein gibt.

	Name	Typ	Was es ist
Ein	Mode	Auswahl	<i>URL</i> (Adresse und Methode ausfüllen) oder <i>cURL</i> (einen fertigen cURL-Befehl einfügen)
Ein	URL	Text oder Message	Die Adresse, z. B. <code>https://api.crossref.org/works</code>
Ein	Method	Auswahl	<i>GET</i> zum Abrufen von Daten (fast immer), <i>POST</i> , <i>PUT</i> , <i>PATCH</i> oder <i>DELETE</i> zum Senden
Aus	API Response	JSON	Die Antwort des Dienstes. Der eigentliche Inhalt steht im Feld <code>result</code> .

Versteckte Einstellungen: *Query Parameters* (die Suchparameter, als JSON-Eingang zum Verbinden), *Headers*, *Body* (für POST), *Timeout* (Sekunden, standardmäßig 30).

Hinweis: Langflow sperrt Adressen auf dem Laptop oder im lokalen Netz, z. B. `localhost`, mit der Fehlermeldung *SSRF Protection: ... resolves to blocked IP address*. Öffentliche Webseiten und die Bibliotheks-Datenbanken funktionieren.

3.2.2 Mock Data

Erzeugt ausgedachte Beispieldaten zum Ausprobieren, z. B. um einen Parser oder eine Loop zu testen, bevor echte Daten da sind. Er hat keine Eingänge.

	Name	Typ	Was es ist
Aus	Result	Table, Message oder JSON	Beispieldaten in der gewählten Form

3.2.3 SQL Database

Führt eine Abfrage in einer SQL-Datenbank aus. Nur sinnvoll, wenn ihr Zugang zu einer Datenbank habt, z. B. einer Kopie eines Bibliothekssystems.

	Name	Typ	Was es ist
Ein	Database URL	Text oder Message	Wo die Datenbank liegt und wie man sich anmeldet, z. B. <code>sqlite:///daten/katalog.db</code>
Ein	SQL Query	Text oder Message	Die Abfrage, z. B. <code>SELECT title FROM books LIMIT 10</code>
Aus	Result Table	Table	Das Ergebnis der Abfrage

Tip: Im Tool Mode kann ein Agent die SQL-Abfragen selbst schreiben.

3.2.4 URL

Lädt den Inhalt einer oder mehrerer Webseiten herunter, auf Wunsch auch den der verlinkten Seiten.

	Name	Typ	Was es ist
Ein	URLs	Text oder Message	Eine oder mehrere Webadressen. Weitere mit dem Knopf + hinzufügen.
Ein	Depth	Schieberegler	1: nur die Seite selbst. 2: dazu jede Seite, auf die sie verlinkt. 3: eine Ebene weiter.
Aus	Extracted Pages	Table	Eine Zeile pro Seite, mit Adresse und Text
Aus	Raw Content	Message	Der Text aller Seiten zusammen, fertig für einen Prompt

Versteckte Einstellungen: *Output Format* (Text, Markdown oder HTML), *Prevent Outside* (auf derselben Website bleiben, standardmäßig an), *Headers*, *Timeout*.

Hinweis: Es gilt dieselbe Adressprüfung wie bei *API Request*. Manche Webseiten, z. B. Wikipedia, lehnen Anfragen ohne Absenderkennung ab. Dann unter *Headers* in der Zeile *User-Agent* eine eintragen, z. B. Bibliothekshackathon (Langflow).

3.2.5 Web Search

Durchsucht das Web, Nachrichten oder einen RSS-Feed, ohne Konto oder Schlüssel.

	Name	Typ	Was es ist
Ein	Search Mode	Auswahl	Web (DuckDuckGo), News (Google News) oder RSS (ein Nachrichten-Feed)
Ein	Search Query	Text oder Message	Die Suchbegriffe. Im Modus RSS heißt das Feld <i>RSS Feed URL</i> und nimmt die Adresse des Feeds.
Aus	Results	Table	Die Treffer mit Titel, Link und einem Textauszug

Versteckte Einstellungen: *Max Results* (standardmäßig 5), *Max Content Length* (behaltene Zeichen pro Treffer), *Language* (hl) und *Country* (gl), z. B. de und DE für deutsche Treffer.

Hinweis: Es gilt dieselbe Adressprüfung wie bei *API Request*. **Tipp:** ein gutes Werkzeug für einen Agenten.

3.3 Models & Agents

3.3.1 A2A Agent

Schickt eine Nachricht an einen anderen Agenten und gibt dessen Antwort zurück: entweder an einen Agenten-Flow in diesem Projekt (*Internal*) oder an einen Agenten irgendwo im Internet (*External*). Für fortgeschrittene Versuche mit mehreren zusammenarbeitenden Agenten.

	Name	Typ	Was es ist
Ein	Mode	Auswahl	<i>Internal</i> (ein anderer Flow in diesem Projekt) oder <i>External</i> (eine Adresse)
Ein	Agent URL	Text oder Message	Adresse des externen Agenten (nur bei <i>External</i>)
Ein	Message	Text oder Message	Was an den Agenten geschickt wird
Aus	Response	Message	Die Antwort des Agenten

3.3.2 Agent

Ein KI-Assistent, der eine Aufgabe Schritt für Schritt bearbeitet und selbst entscheidet, welche **Werkzeuge** er in welcher Reihenfolge nutzt. Die Werkzeuge sind andere Bausteine im Tool Mode, z. B. *Literatursuche* oder *Web Search*.

	Name	Typ	Was es ist
Ein	Language Model	Language Model	Mit welchem KI-Modell der Agent denkt. Hier den Ausgang <i>Language Model</i> von KI-Modell anschließen (siehe oben).
Ein	Agent Instructions	Text oder Message	Rolle und Regeln des Agenten, z. B. „Du bist Recherche-Assistent einer Bibliothek. Erfinde niemals Literatur.“
Ein	Tools	Tool	Die Bausteine, die der Agent nutzen darf. Mehrere lassen sich anschließen.
Ein	Input	Text oder Message	Die Aufgabe, meist aus Chat Input
Aus	Response	Message	Die endgültige Antwort des Agenten
Aus	Structured Response	JSON	Die Antwort als strukturierte Daten (nur mit einem <i>Output Schema</i> , siehe unten)

Versteckte Einstellungen: *Max Iterations* (wie viele Schritte der Agent machen darf, standardmäßig 15), *Current Date* und *Calculator* (eingebaute Werkzeuge, standardmäßig an), *Number of Chat History Messages* (wie viel vom Gespräch sich der Agent merkt), *Output Schema* (eine feste Struktur für die Antwort).

Hinweis: Agenten brauchen Modelle, die gut mit Werkzeugen umgehen. Kleine lokale Modelle scheitern oft daran; nehmt den Cluster. Im Playground könnt ihr jeden Schritt aufklappen und sehen, welches Werkzeug der Agent mit welcher Eingabe genutzt hat.

Beispiele: Flow 04.

3.3.3 Embedding Model

Wandelt Texte in Zahlenreihen („Embeddings“) um, damit sich Texte mit ähnlicher Bedeutung finden lassen. Nur zusammen mit *Knowledge* nötig.

	Name	Typ	Was es ist
Ein	Embedding Model	Auswahl	Anbieter und Modell
Aus	Embedding Model	Embeddings	Das Modell, zum Verbinden mit Bausteinen, die es brauchen

Hinweis: Dafür braucht es einen Embedding-Anbieter, der in diesem Projekt nicht eingerichtet ist, und KI-Modell kann ihn nicht ersetzen. Fragt die Orga, wenn ihr damit arbeiten wollt.

3.3.4 Language Model

Schickt einen Text an ein KI-Modell und gibt die Antwort zurück: die Standard-Version von KI-Modell.

	Name	Typ	Was es ist
Ein	Language Model	Auswahl	Anbieter und Modell
Ein	Input	Text oder Message	Der Text für das Modell, meist ein Prompt
Ein	System Message	Text oder Message	Anweisungen, die für das ganze Gespräch gelten
Aus	Model Response	Message	Die Antwort des Modells
Aus	Language Model	Language Model	Das Modell selbst, für Agent, Batch Run usw.

Versteckte Einstellungen: *Temperature* (0 = sachlich und wiederholbar, 1 = kreativ), *Max Tokens* (höchste Antwortlänge).

Hinweis: Die Anbieter sind in diesem Projekt nicht eingerichtet. Nehmt stattdessen **KI-Modell**: Es hat dieselben Eingänge und Ausgänge.

3.3.5 Message History

Liest frühere Chat-Nachrichten oder speichert neue. So bekommt ein Flow ein Gedächtnis für das Gespräch.

	Name	Typ	Was es ist
Ein	Mode	Auswahl	<i>Retrieve</i> (Nachrichten lesen) oder <i>Store</i> (eine Nachricht speichern)
Aus	Messages	Message	Die früheren Nachrichten als Text, für einen Prompt
Aus	Table	Table	Die früheren Nachrichten als Tabelle

Versteckte Einstellungen: *Number of Messages* (standardmäßig 100), *Sender Type* (nur die der Nutzer:innen, nur die der KI oder beide), *Order* (Reihenfolge).

Tip: Den Ausgang in einen Prompt einsetzen, z. B. **Bisheriges Gespräch:** {verlauf}, damit das Modell sich auf frühere Fragen beziehen kann.

3.3.6 Prompt Template

Schreibt den Text, der an das KI-Modell geht. Teile in geschweiften Klammern wie `{frage}` sind Platzhalter: Für jeden bekommt der Baustein einen Eingang mit demselben Namen, und der angeschlossene Text wird dort eingesetzt.

	Name	Typ	Was es ist
Ein	Template	Text	Die Anweisung mit Platzhaltern, z. B. <code>Antworte kurz: {frage}</code>
Ein	<i>ein Eingang pro Platzhalter</i>	Text oder Message	Der Text, der den Platzhalter ersetzt
Aus	Prompt	Message	Der fertige Text, zum Verbinden mit einem Modell

Versteckte Einstellungen: *Use Double Brackets:* `{{frage}}` statt `{frage}` verwenden, für Vorlagen, in denen einfache Klammern als normaler Text vorkommen.

Tipp: Bei vielen Daten die Daten oben und die Anweisung am Schluss schreiben. **Beispiele:** Flows 00 bis 03.

3.4 LLM Operations

3.4.1 Batch Run

Lässt das KI-Modell **für jede Zeile einer Tabelle** einmal laufen und fügt die Antworten als neue Spalte an. Nützlich, um viele Quellen einzeln zusammenzufassen oder einzuordnen.

	Name	Typ	Was es ist
Ein	Language Model	Language Model	Hier den Ausgang <i>Language Model</i> von KI-Modell anschließen
Ein	Instructions	Text oder Message	Was mit jeder Zeile geschehen soll, z. B. „Fasse dieses Abstract in einem Satz zusammen.“
Ein	Table	Table	Die Zeilen, z. B. aus <i>Literatursuche</i>
Ein	Column Name	Text	Welche Spalte das Modell bekommt. Leer: alle Spalten.
Aus	LLM Results	Table	Die Tabelle mit einer neuen Spalte für die Antworten

Versteckte Einstellungen: *Output Column Name* (Name der neuen Spalte, standardmäßig `model_response`).

Hinweis: Jede Zeile ist ein eigener Aufruf des Modells. Mit einem kleinen lokalen Modell und vielen Zeilen dauert das lange.

3.4.2 Guardrails

Prüft einen Text auf heikle oder unsichere Inhalte, bevor er weitergeht, z. B. personenbezogene Daten, Passwörter oder Versuche, die KI zu manipulieren. Der Flow geht dann bei *Pass* oder *Fail* weiter.

	Name	Typ	Was es ist
Ein	Language Model	Language Model	Hier KI-Modell anschließen (für die KI-Prüfungen nötig)
Ein	Input Text	Text oder Message	Der zu prüfende Text
Ein	Guardrails	Liste	Worauf geprüft wird: <i>PII</i> (personenbezogene Daten), <i>Tokens/Passwords</i> , <i>Jailbreak</i> , <i>Offensive Content</i> , <i>Malicious Code</i> , <i>Prompt Injection</i>
Ein	Checking Method	Auswahl	<i>AI checks</i> , <i>Rules + AI</i> oder <i>Rules only</i> (ohne Modell, findet aber weniger)
Aus	Pass	Message	Der Text, wenn kein Problem gefunden wurde
Aus	Fail	Message	Der Text, wenn ein Problem gefunden wurde
Aus	Result Data	JSON	Einzelheiten zum Gefundenen

Tipp: ein guter Anlass, über Datenschutz zu sprechen, z. B. bevor Anfragen von Nutzer:innen an den Cluster gehen.

3.4.3 LLM Selector

Wählt für jede Eingabe das passendste von mehreren KI-Modellen aus, beurteilt von einem weiteren Modell.
Zum Vergleichen von Modellen; im Hackathon selten nötig.

	Name	Typ	Was es ist
Ein	Input	Text oder Message	Die zu beantwortende Eingabe
Ein	Language Models	Language Model	Die Modelle zur Auswahl (mehrere KI-Modell-Bausteine)
Ein	Judge LLM	Language Model	Das Modell, das entscheidet
Ein	Optimization	Auswahl	Was am meisten zählt: <i>quality</i> (Qualität), <i>speed</i> (Tempo), <i>cost</i> (Kosten) oder <i>balanced</i> (ausgewogen)
Aus	Output	Message	Die Antwort des gewählten Modells
Aus	Selected Model Info	JSON	Welches Modell gewählt wurde
Aus	Routing Decision	Message	Warum es gewählt wurde

3.4.4 Smart Router

Ordnet eine Eingabe mit dem KI-Modell einer von mehreren Kategorien zu, die ihr festlegt, und schickt sie auf dem passenden Weg weiter. Jede Kategorie bekommt einen eigenen Ausgang.

	Name	Typ	Was es ist
Ein	Language Model	Language Model	Hier KI-Modell anschließen
Ein	Input	Text oder Message	Der einzuordnende Text, z. B. eine Anfrage von Nutzer:innen
Ein	Routes	Tabelle	Die Kategorien, eine pro Zeile, z. B. <i>Ausleihe</i> , <i>Recherche</i> , <i>Sonstiges</i> . Jede kann einen eigenen Ausgabertext haben.
Aus	<i>ein Ausgang pro Kategorie</i>	Message	Die Eingabe, auf dem Weg der Kategorie, der sie zugeordnet wurde

Versteckte Einstellungen: *Include Else Output* (ein zusätzlicher Ausgang für Eingaben, die in keine Kategorie passen), *Additional Instructions* (weitere Regeln für die Zuordnung).

Tipp: für eine Auskunft: Fragen zur Ausleihe in einen Flow, Recherchefragen in einen anderen.

3.4.5 Smart Transform

Filtert oder formt Daten nach einer Anweisung in normaler Sprache um, z. B. „nur Publikationen nach 2020 behalten“. Das KI-Modell schreibt dafür ein kleines Programm, das dann auf die Daten angewendet wird.

	Name	Typ	Was es ist
Ein	Language Model	Language Model	Hier KI-Modell anschließen
Ein	JSON	JSON, Table oder Message	Die umzuformenden Daten
Ein	Instructions	Text oder Message	Was geschehen soll, in euren eigenen Worten
Aus	Output	JSON, Table oder Message	Die umgeformten Daten

Hinweis: Das Ergebnis hängt vom Modell ab. Prüft es, besonders bei kleinen lokalen Modellen.

3.4.6 Structured Output

Lässt das KI-Modell in einer festen Struktur mit benannten Feldern antworten, z. B. Autor:in, Jahr und Titel. Ideal, um Angaben aus freiem Text herauszuziehen.

	Name	Typ	Was es ist
Ein	Language Model	Language Model	Hier KI-Modell anschließen
Ein	Input Message	Text oder Message	Der Text, aus dem herausgezogen wird
Ein	Output Schema	Tabelle	Die Felder, eines pro Zeile: Name, Beschreibung, Typ (Text, Zahl, ...) und ob mehrere Werte erlaubt sind
Aus	Structured Output	JSON oder Table	Die herausgezogenen Felder

Versteckte Einstellungen: *Format Instructions* (die Anweisung an das Modell, wie es die Felder füllen soll).

Tipp: z. B. aus jeder Angabe eines Literaturverzeichnisses Autor:in, Jahr, Titel und Zeitschrift herausziehen und eine saubere Tabelle bekommen.

3.5 Files & Knowledge

3.5.1 File System

Gibt einem Agenten einen eigenen Ordner, in dem er Dateien lesen, anlegen und ändern darf. Nur im Tool Mode sinnvoll, verbunden mit einem Agenten.

	Name	Typ	Was es ist
Ein	Workspace Sub-path	Text	Ein Unterordner im Ordner des Agenten. Leer: der ganze Ordner.
Aus	JSON	JSON	Angaben zum Ordner

Versteckte Einstellungen: *Read Only* (der Agent darf Dateien lesen, aber nicht ändern).

Hinweis: Der Ordner liegt im Speicher von Langflow in Docker, nicht im Projektordner auf eurem Laptop.

3.5.2 Knowledge

Speichert Texte in einer **Wissensdatenbank** und durchsucht sie später nach Bedeutung. Das ist die Grundlage für „mit den eigenen Dokumenten chatten“.

	Name	Typ	Was es ist
Ein	Mode	Auswahl	<i>Ingest</i> (Texte speichern) oder <i>Retrieve</i> (suchen)
Ein	Knowledge	Auswahl	Welche Wissensdatenbank
Ein	Input	Message, JSON oder Table	Die zu speichernden Texte, am besten schon mit <i>Split Text</i> zerlegt
Ein	Column Configuration	Tabelle	Welche Spalten durchsucht werden und welche als Zusatzinformation erhalten bleiben
Aus	Results	JSON oder Table	Was gespeichert wurde, oder die passenden Texte

Hinweis: Wissensdatenbanken brauchen einen Embedding-Anbieter (siehe *Embedding Model*), der in diesem Projekt nicht eingerichtet ist. Fragt die Orga, wenn ihr es ausprobieren wollt.

3.5.3 Memory Base

Durchsucht das Langzeitgedächtnis früherer Gespräche mit diesem Flow. Ist *Filter by Session* aus, sucht er über alle Gespräche hinweg.

	Name	Typ	Was es ist
Ein	Memory Base	Auswahl	Welches Gedächtnis durchsucht wird
Ein	Search Query	Text oder Message	Wonach gesucht wird
Aus	Results	Table	Die passenden Teile früherer Gespräche

Versteckte Einstellungen: *Top K Results* (wie viele Treffer, standardmäßig 5), *Filter by Session*.

Hinweis: Eine Memory Base muss zuerst unter **Memories** in der linken Leiste angelegt werden. Außerdem braucht sie einen Embedding-Anbieter, der in diesem Projekt nicht eingerichtet ist.

3.5.4 Read File

Liest eine hochgeladene Datei und gibt ihren Text aus. Funktioniert mit PDFs, Word-Dokumenten, Textdateien, Tabellen und vielen weiteren Formaten.

	Name	Typ	Was es ist
Ein	Files	Datei	Auf Select files klicken und eine oder mehrere Dateien hochladen
Ein	Advanced Parser	Schalter	Liest komplizierte PDFs, Scans und Office-Dokumente genauer (mit dem Werkzeug Docling). Deutlich langsamer und braucht viel Arbeitsspeicher.
Aus	Raw Content	Message	Der Text der Datei(en)

Versteckte Einstellungen: *Separator* (was zwischen mehreren Dateien steht, standardmäßig eine Leerzeile).

Beispiele: Flow 02 (eine Masterarbeit als PDF).

3.5.5 Write File

Speichert Inhalte als Datei, z. B. einen Bericht als Text oder eine Tabelle als Excel.

	Name	Typ	Was es ist
Ein	File Content	Message, JSON oder Table	Was gespeichert wird
Ein	File Name	Text	Name ohne Endung, z. B. pruefbericht
Ein	File Format	Auswahl	<i>csv, excel, json, markdown, txt oder html</i>
Ein	Append	Schalter	An eine vorhandene Datei anhängen, statt sie zu ersetzen (nur Textformate)
Aus	File Path	Message	Wo die Datei gespeichert wurde

Hinweis: Die Datei landet im Speicher von Langflow in Docker, nicht im Projektordner. Um sie in den Projektordner zu kopieren, im Terminal im Projektordner `docker compose cp langflow:<Dateipfad> .` ausführen, mit dem Pfad aus dem Ausgang. Oft ist es einfacher, das Ergebnis in einem Chat Output anzuzeigen und von dort zu kopieren.

3.6 Processing

3.6.1 Data Operations

Ein Werkzeugkasten für Text, JSON und Tabellen in einem einzigen Baustein. Zuerst wählt ihr, welche Art Daten ihr habt (*Input Type*), dann eine Operation. Die passenden Felder und der Ausgang erscheinen erst, wenn eine Operation gewählt ist.

	Name	Typ	Was es ist
Ein	Input Type	Auswahl	<i>Text, JSON oder Table</i>
Ein	Operation	Auswahl	Eine Operation, siehe unten
Ein	Text / JSON / Table	je nach Typ	Die zu bearbeitenden Daten
Ein	<i>weitere Felder</i>		Je nach Operation, z. B. der zu ersetzende Text oder die Spalte zum Sortieren
Aus	Message, JSON oder Table	je nach Operation	Das Ergebnis

Die Operationen:

- **Text:** *Word Count* (Wörter zählen), *Case Conversion* (Groß-/Kleinschreibung), *Text Replace* (ersetzen), *Text Extract* (Text mit einem Muster finden), *Text Head* und *Text Tail* (die ersten oder letzten Zeichen behalten), *Text Strip* (Leerzeichen oder Zeichen an den Rändern entfernen), *Text Join* (zwei Texte verbinden), *Text Clean* (leere Zeilen, doppelte Leerzeichen oder Sonderzeichen entfernen), *Text to DataFrame* (eine Tabelle aus Text in eine echte Tabelle umwandeln)
- **JSON:** *Select Keys*, *Remove Keys* und *Rename Keys* (Felder behalten, entfernen oder umbenennen), *Append or Update* (Felder hinzufügen), *Combine* (mehrere Datensätze zusammenführen), *Path Selection* (einen Wert herausgreifen), *Literal Eval* (Text in strukturierte Daten umwandeln), *JQ Expression* (eine kleine Abfragesprache für JSON, für alles Übrige)
- **Table:** *Filter* (Zeilen behalten, die eine Bedingung erfüllen), *Sort* (sortieren), *Head* und *Tail* (erste oder letzte Zeilen), *Select Columns* (Spalten auswählen), *Drop Column* (Spalte entfernen), *Rename Column* (Spalte umbenennen), *Add Column* (Spalte hinzufügen), *Replace Value* (Wert ersetzen), *Drop Duplicates* (Doppelte entfernen), *Merge* und *Concatenate* (zwei Tabellen zusammenführen)

Tipp: Mit *Table* → *Filter* könnt ihr z. B. aus dem Ergebnis der *Literatursuche* nur die Open-Access-Publikationen behalten.

3.6.2 Dynamic Create Data

Baut einen JSON-Datensatz aus Feldern, die ihr selbst festlegt. Jedes Feld, das ihr in der Konfiguration anlegt, wird zu einem Eingang, den ihr ausfüllen oder verbinden könnt.

	Name	Typ	Was es ist
Ein	Input Configuration	Tabelle	Die Felder, eines pro Zeile: Name und Typ
Ein	<i>ein Eingang pro Feld</i>	je nach Feld	Die Werte
Aus	JSON	JSON	Der Datensatz mit allen Feldern
Aus	Message	Message	Dasselbe als Text

Tipp: z. B. um mehrere Werte (Suchbegriffe, Jahr, Trefferzahl) als Query Parameters an *API Request* zu geben.

3.6.3 Parser

Macht aus JSON oder einer Tabelle Text, nach einer Vorlage. Namen in geschweiften Klammern werden durch die Werte der Felder oder Spalten mit diesem Namen ersetzt. Bei Tabellen wird die Vorlage auf jede Zeile angewendet.

	Name	Typ	Was es ist
Ein	JSON or Table	JSON oder Table	Die Daten
Ein	Mode	Auswahl	<i>Parser</i> (die Vorlage nutzen) oder <i>Stringify</i> (alles unverändert in Text umwandeln)
Ein	Template	Text oder Message	z. B. {titel} ({jahr}), DOI {doi}
Aus	Parsed Text	Message	Der Text, eine Zeile pro Tabellenzeile

Versteckte Einstellungen: *Separator* (was zwischen den Zeilen steht, standardmäßig ein Zeilenumbruch).

Tipp: der richtige Schritt zwischen einer Tabelle und einem Prompt, damit das Modell genau die Spalten bekommt, die es braucht.

3.6.4 Split Text

Zerlegt einen langen Text in kleinere Stücke (Chunks). Nötig für Texte, die für das Modell zu lang sind, oder um eine Liste Stück für Stück abzuarbeiten.

	Name	Typ	Was es ist
Ein	Input	Message, JSON oder Table	Der Text
Ein	Separator	Text	Wo getrennt wird: \n für Zeilen, \n\n für Absätze, . für Sätze
Ein	Chunk Size	Zahl	Höchste Länge eines Stücks in Zeichen (standardmäßig 1000). Kurze Teile werden bis zu dieser Länge zusammengefasst.
Ein	Chunk Overlap	Zahl	Wie viele Zeichen benachbarte Stücke gemeinsam haben (standardmäßig 200), damit an den Rändern kein Zusammenhang verloren geht
Aus	Chunks	Table	Eine Zeile pro Stück, in der Spalte <code>text</code>

Versteckte Einstellungen: *Clean Output* (nur die Textspalte, ohne Zusatzinformationen aus der Eingabe), *Keep Separator* (Trennzeichen behalten).

Tipp: Für genau ein Stück pro Zeile *Separator* auf \n, *Chunk Size* auf 1 und *Chunk Overlap* auf 0 setzen.

3.6.5 Type Convert

Wandelt zwischen Message, JSON und Table um, wenn zwei Bausteine nicht zusammenpassen.

	Name	Typ	Was es ist
Ein	Input	Message, JSON oder Table	Die Daten
Ein	Output Type	Auswahl	<i>Message, JSON oder Table</i>
Aus	Message Output / JSON Output / Table Output	je nach Auswahl	Die umgewandelten Daten

Versteckte Einstellungen: *Auto Parse*: erkennt JSON oder CSV, das als Text vorliegt, und macht daraus echtes JSON oder eine Tabelle.

3.7 Flow Control

3.7.1 Human Input

Hält den Flow an und lässt einen Menschen entscheiden, z. B. einen Entwurf freigeben oder ablehnen. Der Flow geht dann am Ausgang der gewählten Antwort weiter.

	Name	Typ	Was es ist
Ein	Input	Text oder Message	Was sich der Mensch ansehen soll, z. B. der Entwurf
Ein	User Choices	Liste	Die möglichen Antworten, standardmäßig <i>Approve</i> (freigeben) und <i>Reject</i> (ablehnen). Eigene lassen sich hinzufügen.
Aus	<i>ein Ausgang pro Antwort</i>	Message	Die Eingabe, auf dem Weg der gewählten Antwort

Tipp: für „ein Mensch prüft, bevor etwas verschickt wird“, z. B. eine KI-geschriebene Antwort auf eine Anfrage von Nutzer:innen.

3.7.2 If-Else

Vergleicht einen Text mit einem Wert und schickt den Flow auf einen von zwei Wegen: *True* (wahr) oder *False* (falsch). Es ist keine KI beteiligt, das Ergebnis ist also immer gleich.

	Name	Typ	Was es ist
Ein	Text Input	Text oder Message	Der zu prüfende Text
Ein	Operator	Auswahl	Wie verglichen wird: <i>equals</i> (gleich), <i>not equals</i> (ungleich), <i>contains</i> (enthält), <i>starts with</i> (beginnt mit), <i>ends with</i> (endet mit), <i>regex</i> (ein Suchmuster) oder für Zahlen <i>less than</i> (kleiner als), <i>greater than</i> (größer als) und so weiter
Ein	Match Text	Text oder Message	Womit verglichen wird
Aus	True	Message	Der Text, wenn die Bedingung erfüllt ist
Aus	False	Message	Der Text, wenn nicht

Versteckte Einstellungen: *Case Sensitive* (Groß-/Kleinschreibung beachten, standardmäßig an), *Case True* und *Case False* (einen anderen Text statt der Eingabe weitergeben).

Tipp: z. B. nur dann einen ausführlichen Bericht schreiben, wenn das Prüfergebnis „Abweichung“ enthält.

3.7.3 Listen

Empfängt Daten, die ein Baustein *Notify* im selben Flow unter einem Namen abgelegt hat, ohne Verbindungslinie. In Langflow als *Beta* gekennzeichnet.

	Name	Typ	Was es ist
Ein	Context Key	Text oder Message	Der Name, unter dem die Daten abgelegt wurden
Aus	JSON	JSON	Die abgelegten Daten

3.7.4 Loop

Geht eine Tabelle oder Liste **Eintrag für Eintrag** durch. Für jeden Eintrag laufen die an *Item* angeschlossenen Bausteine einmal. Wenn alles durch ist, kommen alle Ergebnisse zusammen bei *Done* heraus.

	Name	Typ	Was es ist
Ein	Inputs	Table, JSON oder Message	Die Einträge, z. B. die Zeilen aus <i>Split Text</i>
Aus	Item	JSON	Der aktuelle Eintrag, für die Bausteine in der Schleife
Aus	Done	Table	Alle Ergebnisse, wenn jeder Eintrag bearbeitet ist

So schließt ihr die Schleife: Der letzte Baustein in der Schleife muss **zurück an den Ausgang *Item*** der Loop angeschlossen werden. Diese Rücklinie sagt der Loop, dass eine Runde fertig ist. Sie wird gestrichelt angezeigt.

Tipp: Eine Schleife lohnt sich, wenn jeder Eintrag mehrere Schritte braucht, z. B. eine API-Abfrage pro Literaturangabe. Für einen KI-Aufruf pro Zeile ist *Batch Run* einfacher.

3.7.5 Notify

Legt Daten unter einem Namen ab, damit ein Baustein *Listen* sie an anderer Stelle im selben Flow abholen kann. In Langflow als *Beta* gekennzeichnet.

	Name	Typ	Was es ist
Ein	Context Key	Text	Der Name, unter dem die Daten abgelegt werden
Ein	Input Data	Message, JSON oder Table	Die abzulegenden Daten
Ein	Append	Schalter	An die unter diesem Namen schon abgelegten Daten anhängen, statt sie zu ersetzen
Aus	JSON	JSON	Die abgelegten Daten

3.7.6 Run Flow

Führt einen anderen Flow aus demselben Projekt als einzelnen Schritt aus. So könnt ihr kleine Flows bauen und kombinieren oder einem Agenten einen ganzen Flow als Werkzeug geben. In Langflow als *Beta* gekennzeichnet.

	Name	Typ	Was es ist
Ein	Flow Name	Auswahl	Der auszuführende Flow
Ein	<i>die Eingänge dieses Flows</i>		Erscheinen, sobald ein Flow gewählt ist
Aus	<i>die Ausgänge dieses Flows</i>		Erscheinen, sobald ein Flow gewählt ist

Tipp: z. B. Flow 01 einem Agenten als Werkzeug geben, damit er Literaturangaben als Teil einer größeren Aufgabe prüfen kann.

3.8 Utilities

3.8.1 Calculator

Berechnet einen Rechenausdruck. Vor allem als Werkzeug für einen Agenten nützlich, weil Sprachmodelle unzuverlässig rechnen.

	Name	Typ	Was es ist
Ein	Expression	Text oder Message	z. B. <code>(17 / 60) * 100</code>
Aus	JSON	JSON	Das Ergebnis

Hinweis: Der Agent hat schon einen Taschenrechner eingebaut (versteckte Einstellung *Calculator*).

3.8.2 Current Date

Gibt das aktuelle Datum und die Uhrzeit aus. Nützlich in Prompts, z. B. für „Publikationen der letzten fünf Jahre“.

	Name	Typ	Was es ist
Ein	Timezone	Auswahl	Die Zeitzone, z. B. <code>Europe/Berlin</code> (standardmäßig UTC)
Aus	Current Date	Message	Datum und Uhrzeit als Text

3.8.3 Python Interpreter

Führt Python-Code aus. Für alle, die ein wenig programmieren können; für die Challenges nicht nötig.

	Name	Typ	Was es ist
Ein	Global Imports	Text	Python-Module, die der Code nutzen darf, durch Kommas getrennt, z. B. <code>math, json</code>
Ein	Python Code	Text oder Message	Der Code. Heraus kommt nur, was mit <code>print()</code> ausgegeben wird.
Aus	Results	JSON	Die Ausgabe

3.9 Bibliothek

Diese Bausteine wurden für den Hackathon gebaut. Sie verbinden Langflow mit Bibliotheks-Datenbanken und kümmern sich um Einzelheiten wie das Zerlegen von Literaturverzeichnissen oder das Warten, wenn eine Datenbank ausgelastet ist.

3.9.1 Katalogsuche (hbz / lobid)

Sucht Bücher und andere Medien im hbz-Verbundkatalog (über lobid.org). Gut für Literatur ohne DOI, die deshalb in Crossref und OpenAlex fehlt.

	Name	Typ	Was es ist
Ein	Suchanfrage	Text oder Message	Titel, Autor:in oder Stichwörter, z. B. Umberto Eco Wie man eine wissenschaftliche Abschlussarbeit schreibt
Ein	Anzahl Treffer	Zahl	Wie viele Treffer (standardmäßig 5)
Aus	Treffer (Text)	Message	Die Treffer als Liste, für einen Prompt
Aus	Treffer (Tabelle)	Table	Die Treffer als Tabelle

Tipp: ein gutes Werkzeug für einen Agenten. **Beispiele:** Flow 04.

3.9.2 KI-Modell

Das KI-Modell dieses Projekts. Es nutzt entweder den Cluster (LiteLLM) oder Ollama auf eurem Laptop, mit Adresse, Schlüssel und Modell aus der `.env`, sodass im Flow nichts eingetragen werden muss.

	Name	Typ	Was es ist
Ein	Input	Text oder Message	Der Text für das Modell, meist aus einem Prompt Template
Ein	System Message	Text oder Message	Anweisungen, die für das ganze Gespräch gelten
Ein	Quelle	Auswahl	<i>Standard (aus .env) (was KI_STANDARD in der .env sagt), Cluster (LiteLLM) oder Lokal (Ollama)</i>
Ein	Modell	Auswahl	Leer lassen für das Standardmodell. Der kleine Pfeil-Knopf lädt die Liste der verfügbaren Modelle.
Ein	Kreativität (Temperatur)	Schieberegler	0 = sachlich und wiederholbar, 1 = kreativ und abwechslungsreich (standardmäßig 0,1)
Aus	Model Response	Message	Die Antwort des Modells
Aus	Language Model	Language Model	Das Modell selbst, für Agent, Batch Run, Structured Output usw. (siehe oben)

Versteckte Einstellungen: *Nachdenken erlauben* („Reasoning“-Modelle vor der Antwort nachdenken lassen: genauer, aber deutlich langsamer), *Maximale Antwortlänge*, *Zeitlimit* (in Sekunden, standardmäßig 600), *Adresse* und *API-Schlüssel* (Adresse und Schlüssel aus der `.env` überschreiben).

Beispiele: alle Beispiel-Flows.

3.9.3 Literaturangaben prüfen (Crossref)

Prüft, ob die Publikationen eines Literaturverzeichnisses existieren und korrekt angegeben sind. Jede Angabe wird bei Crossref nachgeschlagen (per DOI oder als Freitext) und nach Titel, Jahr und Erstautor:in verglichen. Zurückgezogene Artikel werden markiert.

	Name	Typ	Was es ist
Ein	Literaturangaben	Text oder Message	Ein Literaturverzeichnis: nummeriert, durch Leerzeilen getrennt oder eine Angabe pro Zeile. Auch wie in einem PDF umbrochene Zeilen werden erkannt.
Ein	Höchstens prüfen	Zahl	Wie viele Angaben höchstens geprüft werden (standardmäßig 40)
Ein	Auf zurückgezogene Artikel prüfen	Schalter	Auch nach zurückgezogenen Artikeln suchen (standardmäßig an)
Aus	Prüfbericht	Message	Das Ergebnis als Text: zu jeder Angabe  bestätigt,  Abweichung,  nicht gefunden oder  zurückgezogen, mit Einzelheiten
Aus	Tabelle	Table	Dasselbe als Tabelle

Hinweis:  heißt nicht immer „erfunden“: Bücher und Webseiten haben oft keine DOI. Prüft sie mit der *Katalogsuche*. **Beispiele:** Flows 01 und 04.

3.9.4 Literatursuche (OpenAlex)

Sucht wissenschaftliche Publikationen aller Verlage in OpenAlex, mit Abstracts und Links zu freien Volltexten.

	Name	Typ	Was es ist
Ein	Suchanfrage	Text oder Message	Suchbegriffe, am besten auf Englisch, z. B. <code>large language models academic libraries</code>
Ein	Anzahl Treffer	Zahl	Wie viele Treffer (standardmäßig 10, höchstens 50)
Ein	Erscheinungsjahr ab / bis	Zahl	Die Jahre eingrenzen. 0 = keine Einschränkung.
Ein	Nur Open Access	Schalter	Nur Publikationen mit freiem Volltext
Ein	Sortierung	Auswahl	<i>Relevanz</i> , <i>Meistzitiert</i> oder <i>Neueste zuerst</i>
Aus	Treffer (Tabelle)	Table	Die Treffer als Tabelle, z. B. für <i>Volltexte holen</i>
Aus	Trefferliste (Text)	Message	Die Treffer als Liste, für einen Prompt

Versteckte Einstellungen: *Nur mit Abstract* (nur Treffer mit Abstract, standardmäßig an).

Hinweis: Ohne OpenAlex-Schlüssel in der `.env` bremst OpenAlex, wenn viele suchen; der Baustein sucht dann ersatzweise über Crossref. **Beispiele:** Flows 03 und 04.

3.9.5 Literaturverzeichnis finden

Schneidet das Literaturverzeichnis aus einem langen Dokument aus, z. B. aus einer Abschlussarbeit, sodass nur die Angaben weitergegeben werden.

	Name	Typ	Was es ist
Ein	Dokumenttext	Text oder Message	Der vollständige Text, z. B. aus <i>Read File</i>
Aus	Literaturverzeichnis	Message	Nur das Literaturverzeichnis

Beispiele: Flow 02.

3.9.6 Quellen anreichern (OpenAlex)

Ergänzt jede Angabe eines Literaturverzeichnisses um Informationen aus OpenAlex: Abstract, Thema, Zahl der Zitationen und Open-Access-Status. Dazu berechnet er eine Statistik für das ganze Verzeichnis, z. B. den Anteil der Open-Access-Quellen und den Zeitraum.

	Name	Typ	Was es ist
Ein	Literaturangaben	Text oder Message	Ein Literaturverzeichnis, z. B. aus <i>Literaturverzeichnis finden</i>
Ein	Höchstens anreichern	Zahl	Wie viele Angaben höchstens (standardmäßig 60)
Aus	Überblick (Text)	Message	Statistik und alle Quellen als Text, für einen Prompt
Aus	Quellen (Tabelle)	Table	Alle Quellen mit ihren Informationen, z. B. für <i>Volltexte holen</i>

Beispiele: Flow 02.

3.9.7 Volltexte holen (Open Access)

Lädt die freien Volltexte (PDFs) von Publikationen herunter und liest ihren Text aus. Gibt es keinen freien Volltext, wird stattdessen das Abstract genommen.

	Name	Typ	Was es ist
Ein	Publikationen	Table	Die Tabelle aus <i>Literatursuche</i> oder <i>Quellen anreichern</i>
Ein	Höchstens Dokumente	Zahl	Wie viele Publikationen bearbeitet werden (standardmäßig 8)
Ein	Zeichen pro Dokument	Zahl	Längere Texte werden gekürzt, Anfang und Schluss bleiben (standardmäßig 3000). Kleine Modelle vertragen wenig Text.
Aus	Texte für die KI	Message	Die Texte, nummeriert, für einen Prompt
Aus	Tabelle mit Volltexten	Table	Dasselbe als Tabelle

Beispiele: Flow 03.

4. Nicht in dieser Anleitung

- **Legacy-Bausteine:** Langflow blendet alte Bausteine aus, die durch neuere ersetzt wurden. Über das Einstellungssymbol oben in der Leiste lassen sie sich einblenden, sie werden aber nicht gebraucht.
- **Discover more components / Bundles:** weitere Bausteine anderer Anbieter, z. B. OpenAI, Google oder Vektordatenbanken. Die meisten brauchen ein Konto oder einen Schlüssel und sind in diesem Projekt nicht eingerichtet.
- **New Custom Component:** einen eigenen Baustein in Python schreiben. So sind die Bausteine der Kategorie *Bibliothek* entstanden.